CCS_2026

May 8, 2026

1 Simple attacks

This section describes three progressively richer queueing-level models of a shared distributed-quantum-computing (DQC) service: A1, A2, and A3. In all three cases, two logical tenants—an *attacker* and a *victim*—submit entanglement-generation requests to a common service point S. The purpose of these models is not to reproduce full quantum-network physics, but to isolate how *shared service resources* alone can create observable timing interference between co-existing workloads.

1.1 Common Interpretation and Terminology

Before describing the three variants, we define the common objects and timing terms used throughout.

1.1.1 What is being requested?

Each tenant submits a request for an *EPR pair*, i.e., an entangled qubit pair to be distributed across endpoints for later use in a distributed quantum protocol. In a real DQC setting, such pairs are the basic resource behind operations such as teleportation, entanglement swapping, remote state transfer, and distributed gate execution. In the present models, however, the EPR pair is treated only as a *service request*; we do not explicitly simulate the quantum state itself. Instead, we simulate how long the shared infrastructure takes to satisfy such a request.

Thus, when we say "generation," we mean:

> the process by which the shared service attempts to create an entangled pair for a requester.

1.1.2 What is a BSM?

A *Bell-state measurement* (BSM) is a joint two-qubit measurement used in many entanglement-distribution protocols. Operationally, in repeater-like or switching-like architectures, a BSM is often the step that *resolves*, *connects*, or *heralds* entanglement creation between remote users. In these experiments, the BSM is not modeled at the level of measurement operators or output states. Instead, it is abstracted as a *shared processing stage* that consumes nonzero time and, in some variants, can serve only one request at a time.

Hence, when we say a BSM is *serialized*, we mean:

> the shared service point has only one effective BSM processing slot, so requests that are ready for BSM must be handled one-by-one in a queue.

This serialized BSM stage is included because many practical shared entanglement services have a final resolution or coordination step that is less parallel than the front-end generation attempts. The model uses BSM serialization to represent that kind of one-at-a-time shared resource.

1.1.3 What is heralding and what is T_{herald}?

Many entanglement-generation protocols are *heralded*: after an attempt is made, classical information must propagate back to the endpoints or controller to indicate whether the attempt succeeded. This classical acknowledgment or confirmation takes time. We model that delay using a fixed *heralding time*, denoted

$$T_{\text{herald}}.$$

Thus, T_{herald} is present because even after an entanglement attempt succeeds, the requesting tenant does not know immediately; the service completion includes a classical notification delay.

1.1.4 What is T_{attempt}?

Each entanglement-generation attempt takes a finite amount of time. This is represented by

$$T_{\text{attempt}}.$$

Since entanglement generation is probabilistic, one request may require multiple attempts before success. If the number of attempts is denoted N_{att}, then the generation-time portion of service is

$$N_{\text{att}} \cdot T_{\text{attempt}}, \qquad N_{\text{att}} \sim \text{Geom}(p_{\text{success}}),$$

where p_{success} is the success probability of a single attempt.

This term is present because entanglement generation is not deterministic in one shot; repeated trials are often needed.

1.1.5 What is T_{BSM}?

When a model includes a BSM stage, we assign it a fixed service time

$$T_{\text{BSM}}.$$

This is the time for the shared service to perform the final BSM-related processing associated with that request. In the abstract queueing interpretation used here, T_{BSM} represents the occupancy time of the shared BSM resource.

1.1.6 What does "parallel" mean here?

When a stage is called *parallel*, it means multiple requests can be in that stage at the same time. In these experiments, parallelization applies specifically to the *generation stage*: multiple tenants' requests may be undergoing entanglement-generation attempts simultaneously, provided the model allows sufficient shared capacity.

Importantly, this does *not* mean the entire service is parallel. In A3, for example, generation is parallelizable but the BSM stage is still serialized.

1.1.7 What does "serialized" mean here?

A stage is *serialized* if only one request can occupy it at a time. If several requests are waiting for that stage, they must form a queue and be served sequentially. In A1, the entire service is serialized. In A3, only the final BSM stage is serialized.

1.2 Common Description and Assumptions

All three experiments share the following assumptions.

1. There are exactly two logical tenants: an attacker and a victim.
2. Both submit EPR-generation requests to the same shared service point S.
3. The attacker always probes during the experiment, while the victim is toggled *OFF* or *ON*.
4. The attacker and victim are logically distinct, but they are not instantiated as geographically explicit physical network nodes with channels, routing, or repeater placement.
5. The experiments track only *timing*: request issue time, completion time, and latency. No explicit qubit state, fidelity, decoherence, or measurement error is modeled.

6. The offered load is periodic rather than trace-driven: the attacker and victim generate requests according to fixed periods.

7. The common shared point S is the only place where interference occurs.

Therefore, these models should be interpreted as *minimal service abstractions* of shared DQC infrastructure, not as complete network simulations.

1.3 Common Experimental Setup

Each run contains:

- one attacker requester,
- optionally one victim requester,
- one shared service node S.

The attacker and victim submit `EPRRequest` objects to S. Each request records a tenant label, a request identifier, a request timestamp, and a callback to be invoked on completion. The simulation then measures the latency

$$L = T_{\text{done}} - T_{\text{req}}$$

for attacker requests and compares the attacker-latency distribution between victim-OFF and victim-ON cases.

The important point is that the *topology* is intentionally minimal:

$$\text{Attacker requester} \rightarrow S \leftarrow \text{Victim requester}.$$

What changes from A1 to A3 is not the external topology, but the *internal structure of the shared service* S.

1.4 A1: Single Serialized Entanglement Service

1.4.1 Description and Assumptions

A1 is the simplest model. The entire shared service S is represented as one *single-capacity FIFO server*. This means:

- requests arrive at one common queue,
- the server takes the oldest request,
- that request occupies the server until fully completed,
- no other request can make progress through S during that time.

Thus, A1 assumes that the whole entanglement-service path can be collapsed into one effective serialized bottleneck. There is no separate memory model and no internal parallelism.

1.4.2 Setup

The setup consists of one attacker requester and one optional victim requester feeding one shared server S. Both tenants issue EPR requests periodically. The victim can be disabled to create the victim-OFF baseline or enabled to create the victim-ON interference case.

1.4.3 Modeling

For one request in A1, the total service time is modeled as

$$T_{\mathrm{A1}} = N_{\mathrm{att}} T_{\mathrm{attempt}} + T_{\mathrm{BSM}} + T_{\mathrm{herald}}.$$

Each term has a distinct meaning:

- $N_{\mathrm{att}} T_{\mathrm{attempt}}$: time spent repeatedly attempting entanglement generation until success;
- T_{BSM}: time spent in the shared BSM processing stage;
- T_{herald}: time for classical confirmation of success.

In A1, all three are bundled into one uninterrupted server-occupancy interval. Therefore, what is *serialized* is the *entire service*: generation attempts, BSM processing, and heralding are all treated as part of one one-at-a-time path.

The queueing consequence is immediate: if the victim injects more requests, attacker requests wait longer in the FIFO queue before their own service interval begins.

1.5 A2: Shared Memory-Limited Parallel Generation

1.5.1 Description and Assumptions

A2 changes the internal abstraction of S. Instead of assuming one fully serialized server, A2 assumes the dominant shared resource is a *memory pool* of capacity K. This shared memory pool limits how many entanglement-generation processes can be in flight at the same time.

The key assumption is:

> generation attempts can proceed in parallel, but only up to the available shared capacity.

There is no explicit serialized BSM stage in A2. Hence A2 isolates the effect of *finite shared capacity* without introducing a one-at-a-time completion stage.

1.5.2 Setup

The same attacker/victim/off-versus-on structure is retained. The difference is that S now behaves like a resource allocator:

- if memory is available, a request is admitted immediately;
- if memory is full, the request waits;
- once admitted, the request executes its generation process concurrently with other admitted requests;
- on completion, its memory slot is freed.

1.5.3 Modeling

In A2, the service time of one admitted request is

$$T_{\mathrm{A2}} = N_{\mathrm{att}} T_{\mathrm{attempt}} + T_{\mathrm{herald}}.$$

Compared with A1, the BSM term is absent. This is intentional: A2 asks whether *memory occupancy alone* can create timing leakage.

What is *parallelized* in A2 is the *generation stage*. Multiple requests may simultaneously spend time in

$$N_{\mathrm{att}} T_{\mathrm{attempt}} + T_{\mathrm{herald}}$$

as long as memory slots remain available.

What is *not* serialized in A2 is the service itself. There is no single server that all requests must pass through one-by-one. The only source of delay is admission blocking caused by finite capacity K.

Therefore, attacker latency in A2 arises from a different mechanism than in A1:

- in A1, delay is due to FIFO queueing behind one server;
- in A2, delay is due to waiting for a free memory slot.

1.6 A3: Parallel Generation Followed by Serialized BSM

1.6.1 Description and Assumptions

A3 combines the two previous effects into a two-stage model of S.

- **Stage A: Generation stage.** Requests attempt entanglement generation in parallel, subject to a shared memory capacity K.
- **Stage B: BSM stage.** Once generation succeeds, the request enters a queue for a single serialized BSM resource.

This is the richest model because it distinguishes between:

1. *front-end parallelizable work*, and
2. *back-end serialized work*.

The conceptual motivation is that many real shared services can handle multiple ongoing preparation or generation processes, but still contain a coordination, switching, or measurement stage that is less parallel and may become a bottleneck.

1.6.2 Setup

The external setup remains unchanged: attacker and victim submit requests to one shared service node S. Internally, however, S now contains:

- a memory pool of size K,
- a queue of requests waiting to start generation,
- a queue of requests that have finished generation and are waiting for BSM,
- one BSM processing slot.

1.6.3 Modeling

For a request in A3, the service is decomposed into two parts.

Stage A: generation. If memory is available, the request is admitted and occupies one memory slot. It then undergoes entanglement-generation attempts for

$$N_{\text{att}} T_{\text{attempt}} + T_{\text{herald}}.$$

This stage is *parallelizable*: multiple requests may be in Stage A simultaneously, as long as the total number does not exceed K.

Stage B: BSM processing. After Stage A finishes, the request moves to a *ready-for-BSM* queue. It must then wait until the single BSM resource becomes free. Once admitted to that resource, it spends

$$T_{\text{BSM}}$$

in the serialized BSM stage.

Hence the total latency is

$$T_{\text{A3}} = \underbrace{N_{\text{att}} T_{\text{attempt}} + T_{\text{herald}}}_{\text{parallel generation stage}} + \underbrace{W_{\text{BSM}} + T_{\text{BSM}}}_{\text{serialized BSM queue and service}},$$

where W_{BSM} is the queueing delay before BSM.

What is parallelized? The *generation of entanglement requests* is parallelized. That is, several requests can simultaneously be in the attempt/retry process if memory slots are available.

What is serialized? The *final BSM stage* is serialized. Even if several requests finish generation at nearly the same time, only one can undergo BSM processing at once.

Why does memory matter here? A crucial modeling detail is that memory is not released immediately when Stage A finishes. Instead, a request keeps occupying its memory slot until it completes the BSM stage. This couples the two bottlenecks:

- if the BSM is slow, completed requests accumulate waiting for BSM;
- while waiting, they still hold memory;
- holding memory reduces capacity for new generation requests;
- this creates back-pressure from the serialized BSM stage into the supposedly parallel front end.

Therefore, A3 captures both:

1. *admission contention* from finite memory, and
2. *completion contention* from a serialized BSM resource.

1.7 Interpretation of the Three Models

The three variants represent progressively richer abstractions of the same shared DQC service point:

- **A1:** the entire entanglement service is one serialized bottleneck;
- **A2:** the key bottleneck is finite shared capacity for parallel generation;
- **A3:** generation is parallel up to capacity, but final BSM resolution is serialized.

Thus, the progression from A1 to A3 is not a change in network topology but a change in how explicitly the internal service stages are modeled.

1.8 What These Models Capture and What They Omit

These experiments are intended to capture one specific phenomenon:

> whether shared DQC service resources alone can create attacker-observable timing leakage.

They intentionally omit:

- explicit physical endpoints or geographic node placement,
- channel loss and routing,
- explicit Bell states or state fidelity,
- dephasing, decoherence, or readout noise,
- application-level distributed algorithms.

Accordingly, A1–A3 should be understood as *queueing-level service models* for shared entanglement infrastructure. Their purpose is to establish the timing side-channel mechanism first, before introducing more detailed models of state quality or topology.

1.9 Reference DQC Architecture Families: Local Modular (M) and Remote Networked (R)

Our initial models A1–A3 were intentionally designed as *queueing-level* abstractions of a shared entanglement service. Their purpose was to isolate a narrow but important question: whether shared distributed-quantum-computing (DQC) service resources, by themselves, can create observable timing interference between coexisting tenants. Accordingly, those models abstract away explicit endpoint placement, physical channels, routing, state fidelity, decoherence, and application-level distributed protocols, and instead focus only on the timing experienced by entanglement-generation requests. This level of abstraction is appropriate for establishing a proof-of-concept side-channel mechanism, but it is not yet sufficient for a full architectural security claim. To support stronger results, the attack model must be grounded in explicit classes of DQC systems rather than in a topology-free service abstraction alone.

A natural challenge arises immediately: the DQC literature does not yet provide a single universally accepted network architecture. Contemporary simulators and system papers instead reflect a landscape in which multiple architectural styles coexist. For example, NetSquid positions itself as a simulator for both *quantum networks* and *modular quantum computing systems*, spanning the physical layer, control plane, and application level, which already suggests that "distributed quantum computing" covers more than one system organization. Similarly, the SeQUeNCe simulator explicitly notes that the community has not yet converged on a rigid architecture for the quantum internet. In other words, the lack of one canonical blueprint is not a weakness of our formulation; it is an accurate reflection of the state of the field.

For this reason, we do not attempt to anchor the paper to a single vendor-specific or protocol-specific deployment. Instead, we define two broad, source-grounded reference architecture families that together span the design space most relevant to our threat model:

1. **Architecture M: Local Modular DQC**, in which multiple QPUs or quantum modules are integrated within the same larger computing system and cooperate through short-range interconnects and shared control or entanglement services.

2. **Architecture R: Remote Networked DQC**, in which physically separated nodes cooperate over explicit quantum networking infrastructure, typically involving entanglement distribution, heralding, switching, and possibly repeater-like or transduction-assisted links.

We choose these two families for three reasons. First, together they capture the two dominant ways in which "distributed" quantum computation is currently envisioned: *modular scaling within one system* and *networked operation across separated systems*. Second, both families are already visible in existing tools and hardware roadmaps. NetSquid explicitly targets modular quantum computing systems as well as networked quantum systems, and recent modular-architecture literature treats inter-module entanglement generation as a promising path to scaling beyond a single monolithic chip. On the networking side, repeater-style, switch-mediated, and multi-node entanglement-distribution architectures are widely studied, and recent system discussions from IBM describe a progression from on-chip and chip-to-chip couplers to meter-scale couplers and then to longer-range networking through quantum networking units (QNUs). Third, these two families align cleanly with our attack mechanism: both contain shared resources whose occupancy, serialization, or bounded capacity can induce request latency observable by a tenant.

Why not begin from a single fixed architecture? At the current stage of the field, doing so would be artificially restrictive. A paper that assumes one very specific hardware realization risks tying the attack to a narrow implementation detail rather than to the broader systems phenomenon we seek to expose. Our objective is instead to identify a class of vulnerabilities that can arise whenever DQC workloads depend on *shared entanglement infrastructure*. The M/R split lets us remain concrete enough to be architecturally meaningful, while still general enough to avoid overclaiming a nonexistent industry standard.

1.9.1 Architecture M: Local Modular DQC

Architecture M represents DQC systems in which the constituent quantum processors are separate modules, but those modules remain part of one tightly integrated larger system. The modules may reside in the same cryogenic environment, the same rack-scale platform, or the same datacenter-scale installation with tightly

synchronized classical control. The defining property is that the system is *distributed across modules* rather than across broad-area network nodes, yet those modules still rely on explicit inter-module coordination to realize larger computations than a single module can support.

This architecture family is strongly motivated by current modular-scaling roadmaps. In modular hardware, one does not attempt to place all qubits on one monolithic chip indefinitely. Instead, multiple smaller modules are linked together, and high-value resources such as entanglement generation, measurement coordination, calibration state, timing synchronization, and scheduling support become natural points of sharing. IBM's public roadmap illustrates precisely this kind of scaling ladder: long-range couplers within a chip, short-range chip-to-chip integration, and then larger couplers that connect modules at approximately meter scale. Likewise, recent studies on modular, error-corrected architectures emphasize that the performance of such systems depends critically on the quality and rate of inter-module entanglement generation.

In our paper, Architecture M should be understood as the most natural architectural home for the current A1–A3 models. Under M, the attacker and victim are tenants that submit requests to an inter-module entanglement service within the same larger modular quantum computer. The service may be logically centralized even if implemented by distributed hardware components. What matters is that multiple tenants contend for resources that are not private to one module. Those resources may include:

- a bounded pool of quantum memories or in-flight entanglement slots,
- a broker or coordination stage that matches requests and tracks completion,
- a Bell-state-measurement (BSM) or resolution stage that may be less parallel than the front-end generation process,
- classical heralding paths and controller-side completion notifications,
- synchronized inter-module scheduling and admission control.

The security significance of M is straightforward. Although the modules are separate, the system is operated as one larger machine, and therefore its service infrastructure is likely to be shared more aggressively than in a geographically remote setting. This makes local modular DQC a particularly plausible environment for contention-driven timing leakage. In fact, Architecture M is the safest primary architecture for this paper because it requires the fewest speculative assumptions: we do not need to assume long-haul repeaters, internet-scale routing, or fully mature quantum networking stacks. We need only assume that multiple modules cooperate through shared entanglement-oriented services, which is already a central premise of modular scaling.

1.9.2 Architecture R: Remote Networked DQC

Architecture R represents DQC systems in which the cooperating quantum nodes are physically separated and interact through explicit quantum networking infrastructure. Here the relevant system elements are no longer just modules within one tightly integrated platform, but *networked nodes* connected by photonic links, heralded entanglement generation, switching or broker functions, and possibly repeater-like processing stages. In this family, the distributed nature of the system is literal: the nodes are not merely subcomponents of one packaged machine, but participants in a networked computation.

This family is motivated by the mainstream quantum-networking literature as well as by emerging hardware roadmaps. Simulators such as NetSquid are explicitly built to analyze repeater chains, switching/control-plane behavior, and large networks of entanglement-distribution nodes. SeQUeNCe likewise takes a modular view of quantum-network simulation precisely because end-to-end architectures remain unsettled. On the hardware side, IBM's networking roadmap introduces the QNU as an interface between stationary processor qubits and the "flying" qubits used for communication, while also distinguishing short-range couplers from longer-range networking mechanisms and transduction-assisted links.

Under Architecture R, the attacker and victim are best modeled as users or applications whose requests traverse a shared network service layer. That layer may contain:

- node-local quantum memories that buffer partially completed or pending entanglement,
- one or more BSM stations or switching elements,

- classical heralding and acknowledgment channels,
- admission and scheduling logic that allocates network opportunities,
- repeater-like stages or broker nodes that resolve or extend entanglement paths.

The attack surface in R differs from M primarily in *where* the shared bottleneck resides. In M, the bottleneck is naturally interpreted as an internal service of one modular machine. In R, the bottleneck may instead lie in the network infrastructure connecting remote nodes: a switch, a broker, a repeater-stage memory bank, a serialized BSM resource, or a shared entanglement-management service. This setting is more ambitious and more general, but it also introduces more confounding factors, including path diversity, distance-dependent loss, heterogeneous link quality, routing, and potentially richer control-plane policies. For that reason, we treat R as essential to the long-term vision of the paper, but not as the only architecture on which the central timing-leakage claim must stand.

1.9.3 Why M and R are the right two categories

The M/R split is not arbitrary. It captures a real conceptual boundary in how DQC systems are designed, operated, and reasoned about.

First, the two categories separate *modular scaling* from *networked scaling*. In modular scaling, the system is still a single larger computing platform assembled from smaller parts. In networked scaling, the system is an actual quantum network of separated nodes. The engineering constraints, control assumptions, and likely sharing patterns differ substantially between these two cases.

Second, the two categories separate *intra-system shared services* from *inter-node network services*. This distinction matters directly for our threat model. The side channel we study is not caused by the mere existence of qubits at multiple locations; it is caused by the existence of shared resources that mediate entanglement requests. In M those resources are internal to a modular machine. In R they are part of a communication fabric or entanglement-distribution substrate.

Third, M and R provide a disciplined way to generalize the proof-of-concept models without discarding them. A1–A3 do not need to be replaced; they need to be *reinterpreted* as service abstractions that can live under either architecture family. This allows us to preserve the clarity of the queueing mechanism while making the overall paper architecturally credible.

1.9.4 How A1–A3 fit under M and R

Once the M/R architecture families are established, A1–A3 can be presented as *internal service abstractions* that instantiate shared entanglement infrastructure within either family.

A1 under M or R. A1 models the extreme case in which the entire service path behaves as one serialized bottleneck. Under M, this could represent a tightly centralized inter-module broker or one-at-a-time coordination service. Under R, it could represent a shared repeater-stage server, a centralized entanglement broker, or another network-side service point with effectively serialized handling.

A2 under M or R. A2 models a service whose main bottleneck is bounded shared capacity rather than one-at-a-time execution. Under M, this maps naturally to a finite inter-module memory pool or a bounded number of simultaneous in-flight entanglement operations. Under R, it corresponds to shared network memories, bounded buffering, or a finite number of outstanding attempts that can be supported before admission must block.

A3 under M or R. A3 is the richest and most realistic of the three because it separates a parallelizable front end from a serialized completion stage. Under M, this is naturally interpreted as parallel inter-module generation followed by a shared BSM or coordination stage. Under R, it maps to a network setting in which multiple generation attempts can proceed concurrently but completion still depends on a shared switch, broker, or BSM resource. Importantly, A3 also captures back-pressure effects, since memory remains occupied while requests await the serialized completion stage.

This embedding of A1–A3 into M and R is precisely why we adopt the architecture-family approach. It lets the paper make two claims simultaneously: first, that the timing side channel exists as a mechanism at the service level; and second, that the same mechanism can arise in both major classes of DQC systems now being pursued.

1.9.5 Why Architecture M is the primary evaluation target

Although both M and R are important, we select Architecture M as the primary reference point for the main evaluation. The reason is not that R is unimportant, but that M offers the strongest balance of realism, defensibility, and experimental tractability.

M is realistic because modular scaling is already a mainstream path toward larger quantum systems. It is defensible because it requires fewer speculative assumptions about long-distance networking stacks, repeater deployment, routing policies, or heterogeneous infrastructure. It is tractable because the service bottlenecks that matter to our attack model can be represented directly within simulation as shared memory, generation concurrency, and serialized BSM-style completion. In other words, M is the shortest path from our current proof-of-concept abstractions to an architecture-grounded security study.

Architecture R remains crucial as the broader generalization target. It shows that the threat is not limited to one integrated machine and may extend naturally to future remote, service-oriented DQC deployments. However, R should be positioned as the broader architecture family to which the core mechanism plausibly extends, rather than as the only setting in which the paper must immediately deliver fully detailed results.

1.9.6 Summary of the architectural choice

We therefore adopt the following position in the paper. Our current A1–A3 models establish a proof-of-concept timing-leakage mechanism for shared entanglement services. To make the overall study architecturally meaningful, we introduce two reference DQC architecture families:

- **M (Local Modular DQC):** multiple QPUs/modules cooperating within one larger tightly integrated system through shared inter-module entanglement services;
- **R (Remote Networked DQC):** physically separated nodes cooperating through explicit quantum networking infrastructure and shared entanglement-distribution services.

These two families are broad enough to reflect the present state of the field, concrete enough to support a credible threat model, and closely matched to the shared-resource timing mechanism established by A1–A3. We choose them not because they exhaust all future DQC designs, but because together they capture the most important architectural divide for understanding where latency-based side channels are likely to arise.

1.10 Execution and Placement Styles in a Local Modular DQC Architecture

In a local modular distributed quantum computing (DQC) architecture, a quantum workload may be mapped and executed across modules in several distinct ways depending on how the circuit is placed, how execution is coordinated across modules, and how much inter-module communication is required. These categories are useful because they separate the structural form of execution from later questions of resource allocation, scheduling, and security. Below, we summarize the primary execution and placement styles relevant to architecture M.

1. Monolithic local execution. In monolithic local execution, the entire quantum circuit runs within a single module without being partitioned across the modular system. No inter-module communication, entanglement distribution, or remote coordination is required, since all qubits and operations remain local to one hardware island. This case serves as the simplest baseline because performance is determined only by the resources and limitations of the chosen module. It is useful as a reference point for understanding what additional overhead is introduced once modular execution begins.

2. Static distributed execution. In static distributed execution, one circuit is partitioned across multiple modules, and that placement remains fixed throughout the entire runtime of the program. Each module is assigned a subset of qubits or subcircuits in advance, and cross-module dependencies are handled through communication or entanglement-assisted operations. The main characteristic of this style is that the inter-module interaction pattern is largely determined once at compile time and does not change dynamically during execution. This is the most common starting point for modular DQC because it provides a stable and analyzable execution structure.

3. Dynamic distributed execution. In dynamic distributed execution, the workload is still spread across multiple modules, but the assignment of computation to modules may change during runtime. Qubits, subroutines, or execution phases may migrate as the system adapts to load, resource availability, or optimization goals. This allows greater flexibility than static placement, but it also increases orchestration complexity because communication patterns and dependencies can evolve over time. As a result, dynamic execution can improve utilization but typically requires more sophisticated control and scheduling support.

4. Sequential modular execution. In sequential modular execution, different stages of a circuit are executed on different modules in an ordered, stage-by-stage manner rather than concurrently. One module completes a designated portion of the computation before state or classical information is transferred to the next module for continued execution. This style reduces simultaneous cross-module activity but introduces staging overhead and can serialize progress across the architecture. It is most natural when a workload can be decomposed into relatively self-contained phases.

5. Parallel modular execution. In parallel modular execution, multiple parts of the same circuit are executed simultaneously on different modules. This style aims to exploit the modular architecture for concurrency, thereby reducing overall runtime when subcircuits can progress independently for some portion of execution. However, parallelism also increases the need for synchronization and inter-module coordination whenever distributed dependencies arise. Consequently, the benefit of this style depends strongly on how much independent work exists relative to the communication overhead.

6. Hybrid sequential-parallel execution. Hybrid sequential-parallel execution combines both staged and concurrent behavior within the same workload. Some phases of the circuit may run in parallel across several modules, while other phases may require ordered execution or synchronization barriers before progress can continue. This is often the most realistic model for nontrivial modular programs because many circuits naturally alternate between locally parallel computation and communication-heavy coordination points. It captures the fact that modular DQC execution is rarely purely sequential or purely parallel from beginning to end.

7. Lightly distributed execution. In lightly distributed execution, a circuit is mapped across multiple modules but only a small number of operations require inter-module communication. Most of the computational work remains local, and the modular boundary is crossed only occasionally. This leads to relatively low communication overhead and makes the distributed nature of the execution less dominant in the overall runtime. Such workloads are especially useful for studying the lower-overhead end of modular execution and for identifying when partitioning remains beneficial.

8. Heavily distributed execution. In heavily distributed execution, cross-module interactions occur frequently, and a substantial fraction of the workload depends on communication or shared entanglement resources. The modular structure is therefore central to execution behavior rather than incidental, and interconnect performance can significantly affect runtime and correctness. These workloads stress the shared communication and coordination layers much more strongly than lightly distributed ones. They are important because they expose the regimes in which modular DQC overheads and bottlenecks become most visible.

9. Pre-partitioned execution. In pre-partitioned execution, the workload is formulated or compiled with the modular boundaries already known and reflected in the program structure. The circuit is therefore naturally aligned with the target modules, and the amount of additional partitioning work required at compile time is reduced. This style is advantageous when the algorithm itself has an inherent modular decomposition or when the hardware layout is fixed and known in advance. It typically produces cleaner and more predictable inter-module communication patterns than arbitrary post hoc partitioning.

10. Post-partitioned execution. In post-partitioned execution, the workload begins as a monolithic circuit and is later divided across modules by a compiler or partitioning algorithm. The quality of this partition strongly influences the resulting communication demand, since poor cuts can introduce many unnecessary remote interactions. This is an important category because many benchmark and application circuits are originally written without modular boundaries in mind. As a result, post-partitioned execution directly connects compilation quality to distributed-execution overhead.

Architecture justification: Architecture M = local modular superconducting DQC nodes: superconducting transmon compute modules interconnect: short-range cryogenic microwave / coax link topology: shared-star / hub modular architecture

That gives us a stable hardware interpretation for everything that follows:

- workload extraction
- module placement
- remote-operation mapping
- shared-resource modeling

The next step is to define the node model and what exactly the shared hub provides.

2 System Setup

This section defines the baseline system setup used in our study. Since our goal is to vary the workload while keeping the underlying modular architecture fixed, we first establish a single hardware interpretation, a fixed topology, and a clear mapping from circuit-level events to architectural resources. This setup is intentionally defined at the architecture level rather than at the device-physics or pulse level, since the present objective is to study how execution traces from quantum circuits interact with a local modular distributed quantum computing (DQC) system.

2.1 Fixed Architecture

We fix the baseline to a *local modular superconducting DQC* architecture. In this setting, the system is composed of multiple superconducting compute modules connected through a shared interconnect layer. The node type is therefore a *superconducting transmon compute module*, while the interconnect between modules is modeled as a *short-range cryogenic microwave / coax link*. At the topology level, we use a *shared-star / hub modular architecture*, in which all nonlocal interactions are mediated through a common hub rather than through direct module-to-module links.

This choice serves two purposes. First, it gives a stable and realistic local-modular hardware interpretation for the execution traces extracted from quantum circuits. Second, it provides a natural shared-resource structure, which is essential for studying contention, delay, and later interference effects. In particular, a shared hub architecture gives a concrete location where nonlocal operations are serialized, buffered, scheduled, or otherwise constrained by limited common resources.

2.2 Topology

The fixed topology is a star centered around a shared hub module. Each compute module connects only to the hub, and there are no direct compute-module to compute-module links in the baseline system. Thus, if two modules must participate in a nonlocal interaction, they do so only through the shared hub. This yields the basic connectivity pattern

$$\texttt{module}_i \leftrightarrow \texttt{hub}_0,$$

for each compute module $\texttt{module}_i$.

The topology is fixed for the remainder of the setup, meaning that later changes in observed behavior are attributed to differences in workload structure rather than to differences in network organization. This is important because varying both topology and workload simultaneously would make it difficult to isolate the source of contention or execution delay. By keeping the star structure constant, we obtain a clean baseline for comparing different execution styles extracted from real quantum circuits.

2.3 Compute Module Model

Each compute module is modeled as an architectural execution unit representing a small superconducting transmon-based processor block. At this level of abstraction, the compute module is not a pulse-level or Hamiltonian-level model. Instead, it is the entity that owns a subset of qubits, executes local operations on those qubits, and issues remote-operation requests when the trace contains a nonlocal dependency.

Concretely, a compute module contains three conceptual elements. First, it contains a *local qubit register*, namely the set of qubits assigned to that module. Second, it contains a *local execution engine*, which is responsible for executing all operations whose operands lie entirely within that module. Third, it contains a *communication interface* to the hub, which is used whenever a cross-module operation appears in the execution trace.

Accordingly, the compute module can perform any operation that is fully local to its assigned qubits, but it cannot directly complete an operation whose operands span multiple modules. In such a case, it must forward the event to the shared hub for nonlocal handling. For the present phase of the work, this architectural definition is sufficient; we do not require transmon device physics, pulse synthesis, or low-level control simulation. The compute module therefore serves as a logical superconducting processor node that interprets circuit traces and interacts with the hub when required.

2.4 Hub Model

The hub is modeled as the central shared interconnect-service node of the architecture. It is not a compute module and does not represent a local quantum processor in the same sense as the compute modules. Instead, it is the shared architectural resource that receives cross-module requests, arbitrates access to remote-operation resources, and introduces queueing, contention, and service delay into nonlocal execution.

Functionally, the hub performs four roles. First, it acts as the intake point for all cross-module requests generated by compute modules. Second, it contains a *shared service pipeline* through which remote operations are handled. Third, it contains a *resource manager*, which determines whether a request can start immediately or must wait. Fourth, it contains a *scheduler or arbiter*, which decides which pending request is served next when resources are limited.

In the present architectural interpretation, the hub represents the shared microwave/coax interconnect access layer and the shared coordination resource for remote superconducting operations. It is therefore the most natural place to locate shared bottlenecks such as serialized access, finite-capacity buffering, admission delay, and scheduling overhead. These shared-resource effects are what later service models will capture. For now, the hub is fixed as the unique architectural point through which all nonlocal interactions must pass.

2.5 Link Model

Each link represents the short-range cryogenic microwave / coax connection between one compute module and the shared hub. Since the system follows a shared-star topology, links exist only between compute modules and the hub; there are no direct links between compute modules themselves. Thus, all inter-module communication is carried indirectly through module-to-hub and hub-to-module traffic.

At the current level of abstraction, each link is treated as a dedicated architectural path rather than a full physical channel model. Its role is to carry remote-operation requests from a compute module to the hub, and to return completion or coordination signals from the hub back to the module. In the baseline setup, the link may be modeled with fixed latency, and can optionally be extended later to include occupancy or finite bandwidth. However, the initial version of the link is intentionally simple so that the main source of contention remains the hub rather than the links themselves.

This abstraction is appropriate because the purpose of the setup is not to reproduce detailed microwave-link physics, but to establish the path by which cross-module execution demand reaches the shared service layer. The link therefore provides the structural connection needed to map circuit-derived workload traces onto the fixed architecture.

2.6 Meaning of a Cross-Module Circuit Event

A cross-module circuit event is any operation in the circuit whose operands belong to different compute modules under the chosen qubit-to-module assignment. Such an operation cannot be completed entirely within a single compute module, and therefore must be realized through the shared hub and interconnect. This definition is central because it provides the bridge between the extracted trace and the architectural service model.

More specifically, if an operation touches qubits that all belong to the same module, then it is treated as a *local event* and executed directly inside that compute module. In contrast, if an operation touches qubits that belong to different modules, then it is treated as a *cross-module event*. In that case, the operation is converted into a remote-operation request, which is sent from the originating compute module to the hub, serviced through the shared architecture, and only then considered complete.

For example, suppose qubit q_0 is assigned to $\texttt{module}_0$ and qubit q_5 is assigned to $\texttt{module}_1$. Then a gate such as

$$\mathrm{CX}(q_0, q_5)$$

is a cross-module circuit event. In our model, this does not mean that the gate is executed directly across two modules. Instead, it means that the gate induces *nonlocal service demand* on the hub/interconnect layer. Thus, a cross-module circuit event is fundamentally the circuit-level representation of shared architectural resource usage.

2.7 Trace-to-Architecture Mapping

The execution traces extracted from QASM circuits are mapped to the architecture according to the following rule. If a trace event is local, then it is executed directly by the compute module that owns the relevant qubits. If a trace event is cross-module, then it is converted into a hub-mediated remote interaction. In this way, the trace becomes the workload input to the architecture.

This mapping makes the separation of responsibilities explicit. The trace extraction logic determines *what operations occur* and *which of them are nonlocal*. The architecture model determines *how those nonlocal operations are serviced*, namely through compute modules, links, and the hub. This separation is important because it allows us to vary workload structure independently of the hardware topology and resource model.

2.8 Scope of the Baseline Model

The present setup is intentionally architectural rather than fully physical. We do not yet model transmon Hamiltonians, pulse envelopes, exact microwave propagation, or the full quantum state stored inside each module. Instead, we model the system as a fixed local modular superconducting architecture that interprets circuit-derived traces and routes nonlocal operations through a shared service layer. This is sufficient for establishing the workload-to-architecture pipeline that underlies the later contention and interference studies.

Accordingly, the baseline setup is finalized as follows:

- **Architecture type:** local modular superconducting DQC,
- **Node type:** superconducting transmon compute modules,

- **Interconnect type:** short-range cryogenic microwave / coax links,
- **Topology:** shared-star / hub modular architecture,
- **Execution mapping:** local trace events execute inside compute modules, while cross-module trace events are converted into hub-mediated remote requests.

This fixed setup provides the baseline on top of which the remaining service models and workload mappings will be built.

2.9 Attacker Model for Architecture M

Now here, the attacker model is defined specifically for **Architecture M**, namely the fixed local modular superconducting DQC system introduced earlier. Architecture M consists of five superconducting compute modules connected through a shared hub/interconnect layer, where all nonlocal interactions are mediated by the common hub. Accordingly, the attack surface considered here is not a photonic Bell-state-measurement station or a wide-area quantum network endpoint, but rather a *shared superconducting interconnect-service layer* through which cross-module operations are admitted, queued, and serviced. The attacker is therefore modeled as a *co-tenant* of the same modular quantum system, whose actions may influence the timing, contention, and observable behavior of shared architectural resources.

A key principle of the attacker hierarchy is that *all tiers are allowed to run arbitrary jobs.* The attacker is never weakened by limiting the class of jobs they can submit. Instead, tiers differ along two more meaningful axes: *control precision* and *observability.* Control precision determines how effectively the attacker can shape the timing and communication structure of its own workload, while observability determines how much information the attacker can infer from the execution of its own jobs. Thus, weaker attackers are not weaker because they are restricted to simple programs, but because they have progressively less leverage over placement, timing, synchronization, and low-level visibility into shared-resource behavior.

The attacker hierarchy is defined from strongest to weakest. The strongest attacker is granted extensive control over workload placement, timing, and multi-module coordination, whereas the weakest attacker still remains a legitimate co-tenant capable of running arbitrary jobs, but is limited to observing only externally visible properties of its own workload. This structure is especially appropriate for Architecture M because the main side channel and degradation path of interest arises through the shared hub: different levels of control determine how deliberately an attacker can excite the hub, and different levels of visibility determine how effectively the attacker can infer victim behavior from its own observed execution outcomes.

2.9.1 Tier 1: Maximum-Control Co-Tenant Attacker

The Tier 1 attacker is the strongest adversary considered for Architecture M. This attacker can run arbitrary jobs and is assumed to possess extensive control over how those jobs are instantiated on the modular architecture. In particular, the attacker may choose or strongly influence module placement, shape the timing of job submission and internal communication bursts, coordinate activity across multiple attacker-controlled modules, and deliberately engineer high cross-module demand in order to stress the shared hub. Such an attacker can therefore create highly structured load patterns designed either to maximize contention or to probe the timing sensitivity of the victim through the shared interconnect service.

In addition to this control over workload generation, the Tier 1 attacker is granted fine-grained observability over its own execution. It can observe detailed timing behavior of its own requests, including completion delays, request turnaround, and repeated-run timing variation. Depending on the exact experimental framing, this tier may also be assumed to know or estimate structural properties of victim placement or scheduling, though it still does not directly read victim state or violate tenant isolation at the software interface. Conceptually, this tier represents an upper-bound adversary that is useful for stress-testing the vulnerability of Architecture M under worst-case co-tenant interference.

The importance of Tier 1 is methodological: it allows the study to determine whether the shared hub/interconnect can, in principle, carry exploitable timing structure under highly coordinated pressure. Because Architecture M is built around a common remote-operation service layer, a strong attacker with placement and timing control can deliberately align bursts of cross-module traffic with victim-sensitive

execution windows. If any timing-based degradation or inference effect exists in the architecture, Tier 1 is the attacker most likely to expose it.

2.9.2 Tier 2: Strong Cross-Module Co-Tenant Attacker

The Tier 2 attacker remains strong, but is more realistic than Tier 1. Like all tiers, it can run arbitrary jobs. It can also shape its own workload so that it produces significant cross-module communication, thereby exercising the shared hub in a deliberate and repeated manner. The difference from Tier 1 is that it is not assumed to control overall system placement policy or possess privileged knowledge of victim layout. Instead, it operates as a powerful but ordinary co-tenant whose influence arises primarily from its ability to generate targeted hub demand through its own distributed computation.

This attacker can still manipulate the timing of its own jobs to a considerable degree. For instance, it may choose when to submit work, how often to repeat communication-heavy subroutines, and whether to generate bursty or sustained cross-module traffic. It also has detailed observability into the timing of its own jobs, which may include latency distributions, request completion timing, or the response of its own distributed subroutines to shared-hub contention. However, unlike Tier 1, it cannot explicitly dictate victim placement, inspect victim internals, or directly manipulate the hub outside the normal job-submission interface.

Tier 2 is arguably the most natural strong attacker for Architecture M. Because the architecture's key shared resource is the hub/interconnect, a co-tenant that runs communication-heavy distributed workloads is already in a position to induce substantial contention and observe its effects indirectly. This tier is therefore well suited for demonstrating realistic side-channel or denial-of-service style behavior in a modular superconducting DQC setting, while avoiding overly privileged assumptions.

2.9.3 Tier 3: Normal Co-Tenant Attacker

The Tier 3 attacker represents a standard co-tenant in Architecture M. It can run arbitrary jobs and is free to choose the application-level content of those jobs, but it does not possess strong low-level control over how those jobs map into the system beyond the ordinary behavior allowed by the platform. In particular, it cannot finely tune module placement, does not intentionally coordinate aggressive multi-module burst patterns, and does not precisely synchronize its workload with the victim. Instead, it submits ordinary jobs whose local and cross-module behavior emerges naturally from the workload itself.

This attacker still observes the execution of its own jobs, but the level of visibility is more ordinary. It may measure total job completion time, performance slowdown, repeated-run variance, or other high-level timing outcomes available to a tenant using the system. It is not assumed to have privileged instrumentation at the hub or detailed internal service traces. Thus, the Tier 3 attacker is strong enough to be relevant to shared-resource interference, yet weak enough to resemble a realistic user of a modular DQC platform rather than an adversary with specialized orchestration privileges.

Tier 3 is important because it tests whether Architecture M leaks timing structure or suffers measurable degradation even under everyday co-tenancy. If the victim's behavior influences the attacker's own job timing through the shared hub, then even a normal tenant may become an effective observer of workload-induced contention. In this sense, Tier 3 helps determine whether the architecture is vulnerable not only to explicitly malicious probing, but also to realistic multi-tenant interference that may arise from ordinary use.

2.9.4 Tier 4: Weak Co-Tenant with Coarse Output Visibility

The Tier 4 attacker is weaker not because it cannot run sophisticated jobs, but because it has less visibility into what happens during execution. It can still run arbitrary jobs on Architecture M, including workloads that may naturally induce local and cross-module operations. However, its observations are limited to coarse, externally visible properties of its own jobs, such as total runtime, success or failure, final output availability, or aggregate slowdown over repeated runs. It does not obtain fine-grained timing information for individual remote requests or detailed progress-level feedback from the hub service.

This limitation matters because Architecture M's side-channel surface is strongly timing-based. A strong attacker can exploit small variations in queueing and turnaround, whereas a weaker attacker must rely on larger aggregate signals that survive through the entire execution of a job. Tier 4 therefore models a realistic tenant who has access only to the standard outputs and timing behavior that would normally be exposed by

the software stack. Such an attacker can still perform repeated experiments and statistical comparisons, but must do so using much coarser measurements.

Tier 4 is still a credible attack model for Architecture M because the shared hub may create execution-level slowdowns visible at the job boundary, even if fine internal service timing is hidden. If a victim's distributed workload significantly changes the attacker's overall completion time or throughput, then leakage or denial-of-service style degradation may still be observable without privileged instrumentation. This tier therefore captures a weaker but still meaningful class of real-world co-tenant adversaries.

2.9.5 Tier 5: Weakest Realistic Co-Tenant Attacker

The Tier 5 attacker is the weakest, but also among the most realistic for a shared modular system. It can run arbitrary jobs, but it has no special control over placement, synchronization, or low-level timing behavior beyond what follows naturally from ordinary use. Its visibility is restricted entirely to the externally visible outcomes of its own jobs, and it must infer any shared-resource effect from repeated runs and statistical changes in those outputs. It has no insight into victim placement, no direct view of hub state, and no detailed per-request timing measurements.

This attacker is best understood as a noisy-neighbor adversary. It does not possess privileged control or instrumentation; instead, it simply coexists in the same modular superconducting DQC system and observes whether its own jobs behave differently depending on what else is running. In Architecture M, even this weak attacker can still be relevant because nonlocal operations are funneled through a shared hub, and shared bottlenecks can manifest as observable variation in externally visible completion times or other coarse job-level metrics.

Tier 5 is particularly important for framing realistic deployment concerns. If meaningful timing effects or performance degradation are visible even at this weak level, then the architecture may need stronger isolation or scheduling defenses even in ordinary multi-tenant operation. Conversely, if leakage disappears at Tier 5 while remaining visible at stronger tiers, that provides a useful boundary on how much attacker sophistication is required to exploit the shared interconnect of Architecture M.

2.9.6 Summary of the Tiered Attacker Hierarchy

The tiered attacker hierarchy for Architecture M can therefore be summarized as follows. All attackers can run arbitrary jobs, including sophisticated and communication-heavy workloads. What changes from Tier 1 to Tier 5 is the degree of control the attacker has over workload shaping, timing precision, placement awareness, and synchronization, as well as the granularity with which the attacker can observe the behavior of its own jobs. This structure makes the hierarchy well aligned with the architecture itself, since the main attack surface is the shared hub/interconnect, and both exploitability and observability depend directly on how much leverage the attacker has over hub demand and how much timing information it can recover from its own execution.

In the context of this work, the hierarchy also provides a principled progression for evaluation. Stronger tiers are useful for exposing the full timing sensitivity of the architecture and establishing worst-case shared-resource effects. Weaker tiers are useful for assessing whether those effects survive under realistic co-tenant assumptions. Because Architecture M is a local modular superconducting DQC system rather than a photonic or wide-area network architecture, this attacker hierarchy is intentionally framed around shared interconnect-service contention and self-observable timing behavior, rather than around Bell-state-measurement control, probabilistic entanglement retries, or direct network-layer packet observation.

2.10 Attack-Design Knobs for Architecture M

This subsection records the full design space of attack-related knobs for **Architecture M**, namely the fixed local modular superconducting distributed quantum computing (DQC) system used throughout this study. The purpose of this writeup is twofold. First, it establishes a comprehensive inventory of the modeling decisions that may affect attack behavior, timing sensitivity, and interpretability. Second, it serves as a reference point for later phases of the project, so that additional attacker models, placement policies, or scheduling strategies can be introduced in a controlled manner without losing clarity about which assumptions are being changed.

At the present stage of the work, not all of these knobs are being varied. In particular, the underlying architecture and the core workload generation pipeline are intentionally being held fixed so that early results can be attributed to attacker behavior rather than to changes in the system substrate. Nevertheless, the full list is documented here because many of these knobs will become relevant once stronger or more realistic attack models are added. The design space is best understood in five broad categories: *placement knobs*, *workload knobs*, *attack knobs*, *architecture knobs*, and *evaluation knobs*. Each category is described below in detail.

2.10.1 1. Placement Knobs

Placement knobs determine how victim and attacker workloads are mapped onto the five compute modules of Architecture M. Since all nonlocal interactions are mediated through the shared hub, placement strongly affects which module pairs are active, how often the hub is exercised, and whether interference is isolated to the shared interconnect or also includes direct competition inside compute modules. Placement is therefore one of the most important determinants of attack visibility and interpretability.

Victim allocation. The victim allocation specifies which subset of the five compute modules is used by the victim workload and how its logical qubits are distributed across those modules. Possible choices include single-module placement, two-module distributed placement, three-module placement, four-module placement, or full five-module placement. More generally, the victim may be assigned any subset of modules compatible with the chosen execution style. This knob affects the number of cross-module operations induced by the victim circuit, and therefore directly influences baseline hub demand even before any attacker is introduced.

Attacker allocation. The attacker allocation specifies which subset of compute modules is used by the attacker workload. As with the victim, the attacker may be placed in a single module or distributed across multiple modules. This knob determines the attacker's potential to generate local-only versus cross-module traffic, and therefore strongly affects how much hub pressure the attacker can create. In practice, attacker allocation also interacts with workload choice: a communication-heavy attacker is only effective if its qubit mapping produces meaningful cross-module demand under the chosen placement.

Allocation overlap. Allocation overlap determines whether the victim and attacker share compute modules. The main choices are: *disjoint allocation*, in which victim and attacker use separate module subsets; *partial overlap*, in which they share some but not all modules; and *fully unconstrained or overlapping allocation*, in which the attacker may be co-placed wherever the scheduler permits. This knob is especially important because it determines whether interference arises only through the shared hub or also through direct local resource contention within compute modules. For early attack studies, disjoint allocation is often preferred because it isolates the attack surface to the shared interconnect-service layer.

2.10.2 2. Workload Knobs

Workload knobs determine the form of the victim and attacker jobs themselves. These include both the type of execution style used to interpret a circuit and the actual circuit instance being submitted. Although these knobs are being held fixed in the current phase for methodological clarity, they remain part of the broader design space and are recorded here for completeness.

Victim execution style. The victim execution style determines how the victim's QASM workload is mapped into the modular architecture. In the current framework, the primary options are: *monolithic local execution*, in which all operations remain inside one compute module; *static distributed execution*, in which qubits are assigned once to modules and cross-module operations are identified under that fixed placement; *sequential modular execution*, in which the same distributed placement is enriched with stage structure; and *sequential modular v2*, in which stage-aware release and completion logic make timing effects more explicit. This knob affects not only the number of cross-module events, but also how those events are released over time.

Attacker execution style. The attacker execution style specifies how the attacker's own workload is interpreted. In principle, the attacker could run a monolithic job, a static distributed workload, a sequential modular workload, or a more burst-oriented distributed trace specifically chosen to stress the hub. This knob is distinct from attacker tier: all attacker tiers are permitted to run arbitrary jobs, but different execution styles give rise to different forms of hub demand. For example, a monolithic attacker may create little shared-hub pressure, whereas a communication-heavy distributed attacker may generate sustained contention.

Workload choice. This knob selects the actual circuit or family of circuits used by each party. The victim and attacker may run the same circuit, different circuits, benchmark circuits of different sizes, or one realistic application paired with a synthetic stress workload. This knob matters because the circuit itself determines the total number of operations, the qubit interaction graph, and the fraction of gates that become cross-module under the fixed modular mapping. Consequently, changing the circuit can significantly alter hub usage, waiting times, and observable slowdown even if the rest of the attack model remains unchanged.

2.10.3 3. Attack Knobs

Attack knobs define how the attacker behaves as an adversary rather than merely as another workload. These knobs determine the attacker's scheduling pattern, degree of control, intended objective, and available observability. They are the central knobs for the attack study itself.

Attacker tier. The attacker tier captures the degree of control and observability granted to the adversary. In the current hierarchy, all attacker tiers are allowed to run arbitrary jobs; they differ only in how precisely they can shape execution and how much they can observe from their own workload. Tier 1 is the strongest attacker, with maximum control over timing and distributed communication patterns. Lower tiers progressively restrict timing precision, synchronization, or observability, with the weakest realistic tier still able to run arbitrary jobs but only observe coarse outputs from its own execution. This knob is foundational because it defines the adversarial assumptions under which all other attack decisions are interpreted.

Attack schedule. The attack schedule determines when attacker-generated requests arrive relative to the victim workload. Important choices include: *always-on overlap*, in which the attacker continuously runs alongside the victim; *synchronized burst attack*, in which the attacker injects bursts of traffic at selected times; *front-loaded attack*, in which hub pressure is concentrated early in the execution; and *periodic probing*, in which attacker requests are issued at regular intervals to probe timing structure. An adaptive attack, in which the attacker changes behavior based on repeated victim observations, may also be considered in some contexts, but it is less realistic when the victim does not necessarily run the same workload repeatedly. This knob is crucial because the timing structure of the attack often matters as much as the total volume of attacker traffic.

Attack objective. The attack objective specifies what the attacker is trying to achieve. One class of objectives focuses on *degradation*, such as maximizing victim waiting time, increasing victim makespan, or amplifying shared-hub congestion. Another class focuses on *inference*, such as making victim communication phases visible through the attacker's own timing observations. A third class combines both goals, for example by using aggressive hub contention to both slow the victim and expose timing signatures. Explicitly identifying the objective is important because different schedules, placements, and metrics may be appropriate for different attack goals.

Attacker control precision. This knob determines how precisely the attacker can shape the internal timing of its own workload. At the strongest end, the attacker may control fine-grained submission or burst timing. At weaker levels, the attacker may only control job start times or rely on natural timing emerging from its own workload. The control-precision knob is particularly important in Architecture M because the attack surface is timing-based: the ability to deliberately align attacker hub demand with victim-sensitive windows can strongly amplify observable effects.

Attacker observability. This knob captures what the attacker can measure from its own jobs. Possible choices range from detailed per-request timing and latency distributions, to per-job runtime only, to coarse output-level observations such as total completion time or success/failure visible at the application boundary. This knob should not be confused with attacker tier, though it is closely related to it. In practice, the same attacker workload can become much more or less effective depending on whether the attacker observes internal timing structure or only coarse end-to-end outputs.

2.10.4 4. Architecture Knobs

Architecture knobs determine how the underlying system behaves independently of any particular victim or attacker workload. Although these are being held fixed in the current study branch, they are still important to document because later work may revisit them in sensitivity analyses or alternative deployment assumptions.

Hub service configuration. The hub service configuration specifies how the shared interconnect-service layer behaves. Relevant parameters include the number of concurrent transfer slots, deterministic setup latency, deterministic transfer latency, event-tick advancement policy, and, in stage-aware execution modes, inter-stage gaps or barriers. These parameters govern admission, queueing, concurrency, and makespan, and therefore strongly affect all attack-visible timing behavior. In the current phase, the hub has already been upgraded from a trivial FIFO serializer to a more realistic deterministic shared interconnect service with finite concurrency and module-occupancy constraints, and these parameters are being held fixed during attacker-tier studies.

Link configuration. The link configuration determines how each compute module connects to the hub. At the present abstraction level, links are modeled as dedicated short-range cryogenic microwave/coax connections, typically with a fixed latency and no detailed signal model. Future work may vary this knob by adding link occupancy, different per-link latencies, or asymmetric link assumptions, but for now the link abstraction remains fixed so that observed timing effects can be attributed primarily to hub contention rather than to per-link complexity.

2.10.5 5. Evaluation Knobs

Evaluation knobs specify how experiments are compared and what outputs are measured. These knobs determine how attack impact is quantified and presented.

Comparison mode. This knob specifies which execution scenarios are compared against each other. Examples include: victim-only versus victim-plus-attacker, static distributed versus sequential under attack, sequential v1 versus sequential v2 under attack, or direct comparisons across attacker tiers. This knob is important because attack effects are almost always relative rather than absolute. For instance, victim waiting time under attack is only meaningful when interpreted against a victim-only baseline under the same architecture and workload assumptions.

Metrics collected. The metrics knob defines which numerical outputs are used to characterize attack impact. Candidate metrics include victim average waiting time, victim average turnaround time, maximum waiting time, hub makespan, number of completed hub requests, number of requests that experience nonzero waiting, per-stage waiting profiles, and attacker-observed timing distributions. Coarser metrics such as job-level slowdown may be relevant for weaker attacker tiers, whereas finer-grained timing metrics are more appropriate for stronger attackers. The current framework already supports both structural baseline metrics and timing-aware hub metrics, and these will likely form the core of the early attacker evaluation.

2.10.6 Current Fixing Strategy

At the present stage of the project, not all knobs are being exercised. In order to keep the study controlled, the *workload knobs* and *architecture knobs* are being held fixed for the current branch of evaluation. This means that the underlying five-module Architecture M, the hub/interconnect model, and the chosen workload

interpretation branch are not being changed while attacker behavior is introduced. As a result, the main knobs currently left open are the *placement knobs*, the *attack knobs*, and the *evaluation knobs*. This is a deliberate methodological choice: by fixing the architecture and workload pipeline, the study can attribute observed differences more clearly to attacker behavior rather than to confounding changes in the system substrate.

2.10.7 Practical Use of the Knob List

This knob list should be treated as the master design inventory for all future attacker studies on Architecture M. When a new attack experiment is proposed, it should be possible to describe it simply by stating the chosen value of each relevant knob. For example, one attack may be described as: victim on three disjoint modules, attacker on the remaining two modules, static distributed victim execution, static distributed attacker execution, Tier 1 attacker, always-on overlap schedule, degradation objective, current hub configuration fixed, and evaluation against a victim-only baseline using waiting time and makespan metrics. Another experiment may keep all other knobs fixed but change only the attack schedule or observability assumption. In this way, the knob framework not only clarifies current assumptions, but also ensures that future extensions remain systematic and easy to compare.

2.11 Tier-1 Attack Design Under Fixed Five-Node Architecture and Fixed Static Deployment

This subsection describes the attack methodology used in the current evaluation branch. The discussion here is *strictly limited* to the following fixed setting:

- **Fixed architecture:** the five-node instance of **Architecture M**, i.e., a local modular superconducting DQC system with five compute modules connected through a single shared hub/interconnect layer.
- **Fixed deployment style: static distributed execution**. In this branch, qubits are assigned to modules once at the beginning of execution and that assignment remains unchanged throughout the run.
- **Fixed placement class: P1 disjoint placement**, where victim and attacker occupy disjoint compute-module subsets and therefore interfere only through the shared hub.
- **Fixed attacker tier: Tier 1**, the strongest attacker considered in the hierarchy, with maximum control over its own workload shaping and high-detail visibility into its own execution outcomes.

Thus, this section does *not* yet vary the architecture, the deployment mode, the overlap class, or the attacker tier. Instead, it studies a structured family of Tier-1 attacks by varying the *attack schedule* and the *attacker probe circuit* while keeping the architectural substrate fixed.

2.11.1 Architectural Scope of the Present Attack Study

The attack study is performed on the fixed five-module version of Architecture M. The system contains five superconducting compute modules, denoted `module_0` through `module_4`, and a single shared hub that mediates nonlocal operations. All local operations execute inside the assigned compute module. Any operation spanning qubits mapped to different modules is interpreted as a cross-module event and is therefore sent to the shared hub for servicing. The hub enforces deterministic service timing and finite concurrent-transfer capacity, thereby exposing a timing-sensitive shared-resource surface to co-tenant workloads.

Within this fixed architecture, the present study assumes **static distributed deployment**. Accordingly, both victim and attacker qubits are assigned to their respective modules once at the start of execution, and all subsequent cross-module classification is determined relative to that fixed mapping. This is important because the attack effects studied here arise from hub contention under a stable modular placement, rather than from stage-aware release, dynamic remapping, or adaptive orchestration.

2.11.2 Fixed Placement Class: P1 Disjoint Allocation

The current branch uses only the **P1 disjoint placement** case. Under P1, victim and attacker do not share compute modules. Instead, the victim occupies a fixed subset of modules, while the attacker occupies the remaining modules. In the current implementation:

- the **victim** is placed across `module_0`, `module_1`, and `module_2`;
- the **attacker** is placed across `module_3` and `module_4`.

This choice is deliberate. By keeping victim and attacker disjoint at the compute-module level, the study isolates the attack surface to the **shared hub/interconnect service**. That is, any performance degradation or timing side-channel effect observed by the attacker is attributable to shared nonlocal-resource contention rather than to direct local-module occupancy conflicts. This makes the results cleaner to interpret and more tightly aligned with the intended architectural threat model.

2.11.3 Tier-1 Attacker Assumption

The present study uses the **Tier 1 attacker**, the strongest attacker in the hierarchy. Tier 1 is defined by the following assumptions:

- the attacker can run **arbitrary jobs**;
- the attacker can deliberately select **communication-heavy probe circuits**;
- the attacker can choose **when and how** its requests are injected into the shared system;
- the attacker can create **bursty**, **continuous**, or **near-saturation** hub demand;
- the attacker can observe **high-detail outputs from its own execution only**.

Importantly, the Tier-1 attacker is *not* omniscient. It does not directly observe victim internals, victim state, victim outputs, or raw hub state unrelated to its own requests. Instead, it obtains detailed telemetry from its *own* distributed execution, which is sufficient to expose timing variations induced by victim-driven hub contention.

2.11.4 Attacker-Visible Outputs

For this Tier-1 branch, the attacker is assumed to observe both request-level and job-level information from its own execution. These observables are the key mechanism through which the timing side channel is studied.

Request-level observables. For each attacker-generated cross-module request, the attacker can observe or derive:

- request completion count,
- per-request waiting time,
- per-request turnaround time,
- aggregate average waiting time,
- aggregate average turnaround time,
- maximum waiting time,
- number of requests that experienced nonzero waiting.

These metrics quantify how the attacker's own remote operations are delayed by shared-hub contention.

Job-level observables. At the job level, the attacker can observe:

- total attacker job makespan,
- total attacker event count,
- total number of attacker completed remote requests,
- total number of attacker requests that waited.

These metrics capture the coarser performance footprint of victim-induced interference on the attacker's overall job.

2.11.5 Victim Workload in the Current Branch

In the current branch, the victim runs a fixed statically distributed workload derived from a supplied QASM circuit. Under the fixed victim placement on `module_0`, `module_1`, and `module_2`, this workload generates a specific mixture of local and cross-module operations. Because the architecture and victim deployment are fixed, the victim's contribution to hub demand remains structurally stable within this branch, allowing the attacker schedule and attacker probe to be the main variables of interest.

At this stage, the victim circuit itself is still a knob that may be varied in future experiments. However, in the present attack writeup, the architecture and deployment are fixed while the attack design is swept systematically.

2.11.6 Attack Schedule Family: A1–A7

The first axis of variation in the attack study is the **attack schedule**. These schedules determine how the attacker's requests are injected relative to the victim workload. Seven schedules are considered.

A1: Victim-only baseline. This is the no-attacker reference run. The victim executes alone under the fixed static distributed deployment, while the attacker submits no workload. This schedule provides the baseline against which all active attack schedules are compared. Since the attacker does not run, its own observables are identically zero.

A2: Always-on overlap. In A2, the attacker runs continuously alongside the victim for the duration of the run. Attacker and victim requests are interleaved in a persistent overlapping pattern, creating sustained shared-hub contention. This schedule serves as the simplest strong active attack and provides a continuous co-tenant pressure baseline.

A3: Front-loaded attack. In A3, attacker activity is concentrated toward the beginning of the run. The attacker's requests are issued early, ahead of or strongly overlapping the initial portion of the victim execution. This schedule probes whether early hub contention can create elevated delays and whether such early load produces a distinct side-channel footprint.

A4: Back-loaded attack. In A4, attacker activity is concentrated later in the run. Compared with A3, the attacker's requests are shifted toward the back end of the victim execution window. This schedule tests whether late overlap produces behavior distinguishable from early overlap. In the current static implementation, A3 and A4 often produce very similar aggregate attacker-visible behavior, indicating that the present system is more sensitive to total load pattern than to front-versus-back ordering.

A5: Periodic probe. In A5, the attacker injects requests periodically rather than continuously. This produces a lighter-weight probing strategy that is less aggressive than always-on overlap while still allowing the attacker to sample the timing state of the shared hub at regular intervals. This schedule is especially useful as a lower-intensity sensing attack.

A6: Bursty synchronized attack. In A6, the attacker injects concentrated request bursts aligned with communication-heavy portions of the victim trace. This schedule is intended to create stronger contention exactly where the victim is most likely to be using the hub. It therefore acts as a structured high-sensitivity probe and often produces the strongest attacker-visible timing amplification short of full near-saturation behavior.

A7: Saturation attack. In A7, the attacker injects traffic aggressively enough to keep the hub near sustained heavy utilization throughout the active window. This is the strongest schedule in the present family and approximates worst-case contention from the attacker side. It is not merely denial of service; rather, it is also useful as a high-signal timing probe because it magnifies latency sensitivity to victim-induced overlap.

2.11.7 Attacker Probe Family

The second axis of variation in the attack study is the **attacker probe circuit**. Rather than using an arbitrary application circuit for the attacker, the current branch introduces three purpose-built attacker probes. Each probe is designed to exercise the hub in a structured way, while remaining disjoint from the victim at the compute-module level.

The attacker probes are built over two attacker-controlled modules, `module_3` and `module_4`, under a fixed attacker-local qubit map. The goal is to use the attacker as a *sensor* of shared-hub timing behavior rather than merely as a second generic workload.

Probe 1: Cross-module CX chain (`probe_1_cx_chain`). Probe 1 is a repeated, simple, communication-heavy probe built from repeated cross-module two-qubit interactions. Its main properties are:

- structurally simple,
- repetitive,
- dominated by repeated cross-module two-qubit interactions,
- easy to interpret as a timing probe.

This probe is intended to provide a clean balanced baseline. Because it is repetitive and not overly complex, its internal behavior is stable, making it well suited for observing victim-induced delay changes without too much self-generated variability.

Probe 2: Bursty entangling probe (`probe_2_bursty_entangling`). Probe 2 is a stronger, more aggressive communication-heavy probe composed of concentrated cross-module entangling bursts separated by short local regrouping phases. Its main properties are:

- strong cross-module intensity,
- concentrated demand bursts,
- high sensitivity to hub contention,
- stronger stress and larger observed timing shifts than Probe 1.

This probe is intended to maximize attacker-visible timing effects while retaining a structured, interpretable pattern. In practice, it tends to be the strongest of the three probes in terms of average wait, max wait, and job makespan.

Probe 3: Light periodic probe (`probe_3_light_periodic`). Probe 3 is a lower-intensity probe consisting primarily of local operations with occasional cross-module requests inserted periodically. Its main properties are:

- lower average hub pressure,
- smaller absolute delays,
- lighter-weight probing footprint,
- useful as a lower-visibility or less invasive attacker.

This probe is useful for determining whether meaningful victim-induced timing information remains observable even when the attacker is not aggressively stressing the hub.

2.11.8 Permutations and Combinations in the Current Branch

Because the architecture, deployment style, placement class, and attacker tier are all fixed in this branch, the experiment space is determined by the Cartesian product of:

- **7 attack schedules**: A1–A7,
- **3 attacker probes**: Probe 1, Probe 2, Probe 3,
- **1 fixed architecture/deployment setting**: five-node Architecture M with static distributed victim placement under P1 disjoint allocation.

Thus, the current attack matrix contains:

$$3 \times 7 = 21$$

distinct attacker configurations *per victim workload*.
These 21 conditions are:

- Probe 1 with A1, A2, A3, A4, A5, A6, A7,
- Probe 2 with A1, A2, A3, A4, A5, A6, A7,
- Probe 3 with A1, A2, A3, A4, A5, A6, A7.

Each of these conditions is evaluated under the same fixed victim deployment and architecture, and the attacker's own request-level and job-level outputs are collected.

2.11.9 How the Current Attack Matrix Should Be Interpreted

The present attack matrix is best viewed as a *fixed-substrate timing-side-channel sweep*. The architecture is not changing. The deployment style is not changing. The placement overlap class is not changing. The attacker tier is not changing. Instead, the study asks:

> Given a fixed five-node modular superconducting architecture and a fixed static victim deployment, how does the attacker's ability to sense shared-hub timing behavior change as we vary the attack schedule and the attacker probe design?

This framing is important, because it means the current results are intended to characterize the *strength and structure of the side channel itself*, rather than to exhaust the full system design space.

2.11.10 Expected Qualitative Roles of the Schedules and Probes

The schedules and probes are not arbitrary; each plays a distinct qualitative role.

Expected role of the schedule family. The A1–A7 schedule family is intended to span a spectrum from no attack to weak probing to strong synchronized and near-saturation interference:

- A1 gives the no-attacker baseline;
- A5 gives the weakest active probing case;
- A2 provides sustained moderate overlap;
- A3 and A4 explore temporally shifted overlap;
- A6 gives structured high-intensity synchronized interference;
- A7 gives near-worst-case sustained heavy contention.

Expected role of the probe family. The three probes span a spectrum of attacker aggression and timing sensitivity:

- Probe 1 provides a balanced, interpretable baseline probe;
- Probe 2 provides the strongest, most sensitive burst-heavy probe;
- Probe 3 provides the lightest, least invasive probe.

Together, the schedule family and probe family define a clean grid for evaluating how robust and how strong the attacker-visible timing channel is under the fixed architecture.

2.11.11 What the Current Study Does Not Yet Vary

To prevent confusion, it is important to state explicitly what is *not yet* varied in the current branch.

- The architecture is fixed to the five-node Architecture M instance.
- The victim deployment style is fixed to static distributed execution.
- The overlap class is fixed to P1 disjoint placement.
- The attacker tier is fixed to Tier 1.
- The hub parameters and link assumptions are fixed.
- The current branch does not yet sweep partial-overlap or victim-heavy overlap placements.
- The current branch does not yet include sequential-v2 attack runs.
- The current branch does not yet use multiple victim circuits within the same presented attack matrix.

2.11.12 What This Attack Matrix Already Establishes

Even with these fixed choices, the current branch already establishes several important points:

1. A co-tenant attacker can extract meaningful timing information from its *own* execution alone.
2. The attacker-visible timing signature depends strongly on the attack schedule.
3. The attacker-visible timing signature also depends strongly on the chosen attacker probe.
4. The side channel exists even when attacker and victim are disjoint at the compute-module level and only meet at the shared hub.

These points make the current study a valid architectural timing-side-channel baseline for fixed static deployment.

2.11.13 Recommended Missing Clarifications and Next-Step Extensions

The current branch is already well defined, but a few clarifications or future extensions should be noted explicitly.

1. Multiple victim circuits. At present, the architecture and deployment are fixed, and the attacker probes and schedules are swept. However, to demonstrate *meaningful leakage* rather than only generic contention sensitivity, the next logical extension is to repeat this attack matrix over multiple victim circuits. This would allow the study to determine whether different victim workloads produce distinguishable attacker-visible timing signatures.

2. Sequential-v2 victim deployment. A natural next extension is to repeat the same Tier-1 probe-and-schedule sweep under fixed **sequential-v2** deployment. This would test whether stage-aware victim release produces stronger or more structured timing leakage than the static case.

3. Additional overlap classes. The present branch is restricted to P1 disjoint allocation. Future work should revisit:

- P2 partial overlap,
- P3 victim-heavy overlap,

to determine how shared-module interference interacts with shared-hub interference.

4. Lower attacker tiers. The current branch uses only Tier 1. Future work should examine whether the same side channel remains observable under weaker but more realistic attacker tiers with reduced control and coarser observability.

5. Victim-side metrics. The current attack analysis is attacker-centered. For completeness, future studies may also report victim degradation metrics alongside attacker-visible metrics, especially when discussing the distinction between denial of service and information leakage.

2.11.14 Summary

In summary, the present attack branch studies a **fixed five-node Architecture M** under **fixed static distributed deployment**, **fixed P1 disjoint placement**, and a **fixed Tier-1 attacker model**. Within this fixed substrate, the attack space is swept across:

- seven attacker schedules (A1–A7), and
- three purpose-built attacker probes.

This yields 21 attack conditions per victim workload, all evaluated using attacker-visible request-level and job-level observables. The resulting methodology forms the current baseline attack framework for Architecture M under static deployment and provides a systematic starting point for later extensions involving multiple victim circuits, sequential-v2 deployment, alternate overlap classes, and weaker attacker tiers.

2.12 Progressive Refinement of the Tier-1 Static Attack Design

In this work, the attacker study did not emerge from a single fixed design choice. Instead, the current attack configuration was reached through a deliberate sequence of refinements motivated by intermediate observations. This progression is important to document, because it explains why the final attack methodology differs substantially from the earliest baseline experiments. In particular, the evolution of the attack design clarifies how we moved from a generic co-tenant contention study toward a more meaningful victim-fingerprinting setup under the fixed five-node Architecture M and fixed static distributed deployment.

2.12.1 Initial Goal and Starting Assumption

The original goal of the attack study was to determine whether a co-tenant attacker could infer meaningful information about a victim workload through shared-resource timing effects in Architecture M. Since the architecture is modular and all nonlocal interactions are mediated through a shared hub/interconnect service, the initial intuition was straightforward: if different victim algorithms create different hub-pressure patterns, then an attacker running its own distributed workload should observe those differences indirectly through its own execution delays.

At the outset, we fixed the architectural substrate in order to keep the study controlled. Specifically, we fixed:

- the **five-node instance of Architecture M**,
- the **static distributed deployment model**,
- the **P1 disjoint placement** class, in which victim and attacker occupy disjoint compute-module subsets and therefore interfere only through the shared hub,
- and the **Tier-1 attacker** assumption, which grants maximum control over the attacker's own workload and high-detail visibility into the attacker's own outputs.

With these choices fixed, the study initially varied the *attack schedule* while using a single attacker workload design.

2.12.2 Stage 1: Baseline Schedule Sweep with a Fixed Attacker Workload

The first attack experiments treated the attacker as a distributed co-tenant running a fixed workload under several different *schedules* relative to the victim. These schedules were denoted A1–A7 and included:

- A1: victim-only baseline,
- A2: always-on overlap,
- A3: front-loaded attack,
- A4: back-loaded attack,
- A5: periodic probe,
- A6: bursty synchronized attack,
- A7: saturation attack.

The main purpose of this first sweep was not yet victim identification, but rather to determine whether the shared hub exposed any measurable side-channel surface at all. The schedule family successfully demonstrated that attacker-visible timing metrics such as average wait, turnaround time, maximum waiting time, and attacker job makespan changed substantially depending on how the attacker overlapped with the victim. This established the existence of a timing-sensitive shared-resource surface.

However, a more important observation soon emerged: although the attacker could clearly distinguish *schedule strength*, it was much less successful at distinguishing *victim type*. In other words, the schedule family strongly separated light, moderate, and heavy attacker contention patterns, but did not yet produce robust victim-class fingerprints.

2.12.3 Stage 2: Realization That the Attacker Was Masking the Victim

When the same attack schedules were tested against multiple victim circuits, including square-root, Bernstein–Vazirani, and QFT-style workloads, the resulting attacker-visible outputs were often remarkably similar. This was especially true under the stronger schedules such as A2, A6, and A7. The key realization was that the attacker's own workload and schedule were often dominating hub behavior to such an extent that victim-specific differences were being partially masked.

Conceptually, the early attacker was behaving more like a *hammer* than a *sensor*. It was capable of proving that the shared hub could be stressed and that attacker-visible delays existed, but it was not yet an especially good mechanism for extracting subtle victim-specific signatures. This was an important turning point in the study. It suggested that if the goal was to identify coarse victim structure rather than merely demonstrate denial-of-service-style degradation, then the attack needed to become lighter, more structured, and more probe-like.

2.12.4 Stage 3: Introduction of Purpose-Built Attacker Probes

To address this issue, the attacker workload was restructured into a family of dedicated *probe circuits* rather than a generic application-like attacker trace. Three attacker probes were introduced.

Probe 1: cross-module CX chain. This probe consisted of repeated simple cross-module two-qubit interactions interleaved with light local operations. It was intended to be structurally simple, repetitive, and easy to interpret.

Probe 2: bursty entangling probe. This probe used denser blocks of cross-module entangling interactions and was designed to be the strongest, most communication-heavy probe.

Probe 3: light periodic probe. This probe used mostly local operations with only occasional cross-module interactions, making it the weakest and least invasive attacker probe.

The motivation behind this probe family was to separate two goals that had previously been conflated:

- **stress testing the hub**, and
- **sensing victim-dependent behavior**.

Probe 2 was intended to maximize sensitivity through stronger hub pressure, while Probe 3 was intended to act as a lightweight sensor of victim-induced modulation.

2.12.5 Stage 4: Comparison of the Three Probe Families

Once the three probes were evaluated across the original schedule family, a much clearer pattern emerged. Probes 1 and 2 consistently produced the strongest delays, confirming that they were effective stress probes. However, because they drove the hub more aggressively, they often continued to blur victim-specific differences. Probe 3, by contrast, frequently produced smaller absolute delays, but those delays appeared to preserve more of the victim's natural timing structure.

This produced the first key insight of the refinement process:

> A stronger attacker probe does not necessarily produce a stronger victim fingerprint. A lighter probe may reveal more victim-specific information precisely because it perturbs the shared hub less.

Thus, while Probes 1 and 2 remained valuable for demonstrating strong interference and queue buildup, Probe 3 became the most promising candidate for victim differentiation.

2.12.6 Stage 5: Sparse-Bursty Sweeps Around the Light Probe

The next refinement step focused on making the attacker both *sparse* and *meaningfully bursty*. The motivation here was to preserve the victim sensitivity of Probe 3 while still allowing some structure in the attacker's interaction pattern. Rather than using the heavier A6/A7-style attack schedules, the study moved toward an A5-like light periodic schedule and then introduced controlled burst sweeps. These sweeps varied:

- the burst size, and
- the spacing between burst injections.

This stage led to another useful insight. For Probes 1 and 2, increasing burstiness generally increased contention in a mostly monotonic manner, with intermediate strong settings often being the most effective. For Probe 3, however, the opposite sometimes occurred: stronger burst grouping could reduce waiting rather than increase it, suggesting that the burst pattern itself interacted with the victim and hub timing in a nontrivial way. This reinforced the view that Probe 3 was behaving differently from the heavier probes and should be treated as a distinct sensing family rather than as merely a weaker stress probe.

2.12.7 Stage 6: Probe-3 Rate Sweep

The final major refinement step in the current branch was to specialize fully on Probe 3 and vary its *probe rate*. Instead of changing the attacker schedule across many qualitatively different attack families, the schedule was fixed to a light A5-like periodic injection policy, and Probe 3 itself was parameterized by how often it issued its cross-module probe operation. Several probe-rate settings were tested, ranging from dense to very sparse.

This step proved to be the most informative so far. Unlike the earlier experiments, the Probe-3 rate sweep showed clearer victim-dependent separation. In particular, the denser rate settings produced different attacker-visible waiting patterns across victim workloads such as BV, QFT, SAT, square-root, and DNN-style circuits. At the same time, once the probe became too sparse, the waiting signal frequently collapsed to zero, indicating that the probe had become too weak to observe meaningful victim modulation. This yielded a second key insight:

> Victim fingerprinting requires a probe that is neither too strong nor too weak. If the probe is too strong, it masks the victim. If it is too weak, the observable waiting signal collapses.

Under the current static deployment, the denser Probe-3 settings appear to strike the best balance between perturbation and sensitivity.

2.12.8 What the Refinement Process Has Taught Us

The trial-and-error process was therefore not incidental; it was necessary to uncover the actual structure of the timing side channel. Several important lessons emerged:

1. **Schedule strength alone is not sufficient.** Heavy schedules such as saturation or synchronized burst attacks are useful for exposing the existence of shared-resource contention, but they are not automatically optimal for victim fingerprinting.
2. **Attacker workload design matters as much as attacker schedule.** A poorly chosen attacker workload may overwhelm the hub and erase victim-specific signatures, even if it is highly effective at causing slowdown.
3. **A light probe can outperform a heavy probe for inference.** Probe 3 demonstrated that smaller perturbation can preserve more of the victim's natural communication behavior.
4. **Burstiness and probe rate must be tuned jointly.** Meaningful side-channel extraction does not come from indiscriminately increasing attacker pressure; rather, it comes from carefully choosing when and how often the probe interacts with the shared hub.
5. **Victim class differentiation is possible, but only under the right operating regime.** The most promising evidence of victim-specific behavior emerged only after moving away from the strongest attacker configurations and toward a refined light probing family.

2.12.9 Current Status of the Attack Methodology

At the present point in the project, the attack methodology has therefore evolved into a more targeted structure:

- the architecture remains fixed to the five-node Architecture M;
- the deployment remains fixed to static distributed execution;
- placement remains fixed to P1 disjoint victim-attacker allocation;
- the attacker remains Tier 1;
- the current most promising branch is the **Probe-3 family with controlled rate variation under a light periodic schedule**.

This is the current best candidate for extracting victim-dependent timing fingerprints under the static deployment model.

2.12.10 What Is Still Missing

Although this refinement process has already produced a much stronger attack methodology than the initial baseline sweeps, several points remain open:

- The current results still support only *coarse victim-class inference*, not exact algorithm identification.
- The study has so far focused on static deployment only; it remains to be seen whether sequential-v2 victim deployment amplifies victim-dependent timing structure.
- The current branch still uses a fixed overlap class (P1 disjoint); partial-overlap and victim-heavy overlap cases may expose stronger or qualitatively different leakage.
- The current analysis has primarily focused on attacker-visible timing; further work may formalize victim classification more explicitly using statistical separation or simple classifiers.

2.12.11 Summary

In summary, the current attack design was reached through a progressive process of refinement. The study began with generic co-tenant schedule sweeps, established the presence of a timing side channel, discovered that heavy attacker traffic could mask victim-specific differences, introduced a family of dedicated attacker probes, and ultimately converged on a light probe-rate sweep as the most promising direction for victim fingerprinting. This evolution is important because it shows that the final attack design was not assumed from the beginning; rather, it was derived through systematic empirical adjustment under a fixed architectural substrate. As a result, the current attack methodology is both more principled and better matched to the goal of extracting meaningful victim-dependent information from the shared hub of Architecture M.

2.13 Attack Development After the Initial Probe-Rate Sweep

After the initial Probe-3 rate sweep, the attack study moved from a broad proof-of-concept phase into a much more deliberate refinement phase. The central question was no longer merely whether a co-tenant attacker could observe shared-resource interference, but rather how the attacker should be designed so that the observed interference became *victim-informative*. In other words, once the rate sweep indicated that Probe 3 under dense or medium probe rates was more promising than the heavier attacker schedules, the remaining effort focused on determining which specific probe design choices preserved victim-specific information, which choices washed it out, and which choices made the attack useful for workload fingerprinting rather than simple contention detection.

2.13.1 Starting Point After the Probe-Rate Sweep

The probe-rate sweep established the first major refinement result. Among the candidate Probe-3 rate settings, the denser probe configurations were consistently more informative than the sparse ones. In particular, the strongest operating points were the dense and medium settings, which were thereafter treated as the most promising branches, while the lighter settings were retained only as weak controls. The key lesson from this phase was that Probe 3 could already reveal victim-dependent differences, but only when the probe rate was large enough to generate a measurable signal. If the probe rate was too sparse, the waiting-time signal often collapsed and the victim workloads became difficult to distinguish.

This phase also clarified the role of Probe 3 conceptually. Unlike the heavier attacker probes, which behaved like stress workloads and often masked victim-specific structure through their own self-generated hub pressure, Probe 3 behaved more like a lightweight sensor. Its value was not in maximizing disruption, but in producing a smaller and more faithful signal that remained shaped by the victim's own communication behavior. That insight guided the remainder of the refinement process.

2.13.2 Fixing the Best Probe-Rate Candidates

Once the dense and medium rate settings had been identified as the useful operating points, the study stopped treating rate as the only knob and instead fixed the best rates in order to vary other aspects of the probe. The reasoning was methodological: if all knobs were allowed to vary simultaneously, it would be impossible to tell whether changes in victim distinguishability came from rate, spacing, duration, or some interaction effect. Therefore, the strategy became one of *independent sweeps*. First, the two strongest rate settings were fixed. Then, with the rate effectively controlled, the spacing between cross-module probe gates was varied to determine whether the same overall probe intensity could produce different victim fingerprints depending on temporal arrangement.

2.13.3 Spacing Sweep at Fixed Dense and Medium Rates

The next phase introduced spacing sweeps for the best dense and medium rate settings. The idea here was that two probes with the same average rate of cross-module interactions might still differ in victim sensitivity if those interactions were arranged differently in time. To test this, four spacing patterns were studied at both dense and medium rates:

- uniformly spaced probes,
- paired probes,
- clustered probes,
- and alternating short-gap/long-gap probes.

The dense-rate spacing sweep showed that the probe spacing did matter, but the result was not what might naively have been expected. Uniform dense spacing and paired dense spacing behaved almost identically across the victim set, indicating that the current paired construction was effectively redundant with uniform probing in the dense regime. By contrast, clustered dense spacing and alternating dense spacing consistently weakened the attacker-visible signal, reducing average waits and compressing the differences among victim workloads. This meant that irregularity in spacing did not improve victim sensing. Instead, it reduced sensitivity by making the probe less effective as a consistent sampler of victim behavior.

The medium-rate spacing sweep told a similar story, although with a little more variation across patterns. Uniform medium spacing again emerged as the safest and strongest default. Paired medium spacing remained very close to uniform spacing. Clustered medium spacing consistently weakened the signal, and alternating medium spacing produced mixed effects: it sometimes approached uniform performance, but did not do so reliably. The overall lesson from both spacing studies was therefore straightforward: the simple uniform arrangement remained the best primary spacing policy, and there was little evidence that more irregular temporal organization improved victim discrimination.

This was a valuable negative result. It meant that the attack did not need additional temporal ornamentation at the spacing level. The cleanest probe remained the best one: Probe 3 with dense rate and uniform spacing.

2.13.4 Victim Structural Saturation Check

Before moving to the next refinement knob, the study paused to examine the victim circuits themselves under the fixed static mapping. This was done through a structural saturation check, where each victim workload was analyzed in terms of its local-only versus cross-module operations. The purpose was to understand whether fixed attacker durations were being applied fairly across victims, or whether some victims were being over-probed while others were being under-probed.

This analysis revealed that the workloads differed substantially in their cross-module structure. Square-root and SAT had high cross-module densities, QFT was moderate, BV was lower, and DNN had very low cross-module density. This explained an important ambiguity in the earlier absolute experiments: if all victims were probed for the same absolute duration, then short victims with relatively few cross-module opportunities could appear artificially heavy simply because the attacker was effectively overextending its observation window relative to the victim's own communication footprint. Conversely, larger victims with many more cross-module opportunities could be under-sampled by the same absolute probe duration.

This structural check motivated the next major design decision: duration should be studied in two different ways, first blindly in absolute terms, and then more carefully relative to each victim's own cross-module complexity.

2.13.5 Absolute Duration Sweep

The first duration experiment was intentionally blind. The same best probe (Probe 3, dense rate, uniform spacing) was run for several absolute numbers of rounds, specifically short to long durations such as 10, 20, 50, 100, and 200 rounds. The purpose of this sweep was not yet fairness, but simple trend discovery: if the attacker observes the victim for progressively longer windows, how does the signal evolve?

This sweep showed that longer probe duration predictably amplified the attacker-visible signal. Average waiting time, maximum waiting time, waited-request count, and job makespan all increased with duration. More importantly, victim separation became clearer as the probe duration increased. Very short durations such as the smallest tested windows gave only weak or partial separation; intermediate durations began to reveal stable victim ordering; and long durations produced strong separation but at the cost of becoming a much heavier and more intrusive attacker.

However, the limitation of the absolute-duration approach became obvious immediately. Since the victims had different cross-module counts, the same absolute duration was not sampling comparable fractions of victim communication behavior. For example, a fixed long duration could cover all relevant cross-module behavior of one victim but only a small portion of another. Therefore, although the absolute sweep was useful in showing that duration mattered, it did not by itself provide the most scientifically meaningful comparison across workloads.

2.13.6 Relative Duration Sweep

To correct this, the next experiment scaled the probe duration relative to the victim's own cross-module operation count. This was a key turning point in the study. Instead of saying "the attacker probes for N rounds regardless of the victim," the question became: what happens if the attacker probes for 10%, 20%, 50%, or 100% of the victim's cross-module opportunity count?

This relative-duration framework produced the clearest fingerprinting results up to that point. It removed the bias of over-probing small victims and under-probing large ones, and the resulting separation became much more interpretable. Under this normalization, the victims sorted more naturally into communication-intensity classes. Square-root and QFT emerged as the strongest interferers under relative comparison, SAT occupied a middle region, BV appeared lighter, and DNN remained the most unusual case because it often looked light in waiting-time terms at short relative windows but large in total attacker makespan. This showed that DNN was not simply a weaker version of the other workloads, but a qualitatively different workload whose effect on the attacker was expressed more in long-run timeline stretch than in immediate queue buildup.

This relative-duration sweep was one of the most important refinements in the entire attack-development process. It transformed the attack from a somewhat ad hoc probing mechanism into one whose observation window was scaled to victim structure. At this point, the attack could be described as a meaningful victim-fingerprinting mechanism rather than a generic contention experiment.

2.13.7 Temporal-Scale Interpretation: Short, Medium, and Long Windows

After the relative-duration sweep had identified the most useful relative windows, the next refinement step was interpretive rather than exploratory. Instead of continuing to search blindly across more parameter values, the existing relative-duration windows were grouped into short, medium, and long temporal scales. The motivation was to understand *what kind of victim behavior* was being sensed at different observation horizons.

The short-timescale study used only the smallest relative windows. These short windows showed that the attacker could already distinguish victims coarsely at early time scales. Even relatively brief probing windows were enough to reveal that square-root and QFT produced stronger early contention signatures than SAT or BV, while DNN remained nearly invisible in waiting-time space at the shortest windows. This indicated that some workloads reveal themselves quickly, through short-window queue effects, while others do not.

The medium-timescale study used intermediate relative windows. This regime turned out to be one of the most useful overall. It produced significantly stronger victim separation than the short windows, but without yet becoming as intrusive as the longest windows. The medium windows appeared to capture something like phase-level victim behavior: not just bursty local congestion, but the structure of a meaningful portion of the workload. At this point, the workload classes became much more distinct, and the attack could rank them with higher confidence.

The long-timescale study used the largest relative windows. This regime produced the strongest raw separation in the victim set. By observing a large fraction, or effectively the full fraction, of the victim's cross-module behavior, the attacker could maximize class separation. However, this came at the cost of becoming a heavier and less stealthy sensing workload. Conceptually, the long-timescale regime measured coarse whole-workload intensity rather than local or phase-specific signatures.

Taken together, these temporal-scale studies gave the attack a much richer interpretation. The attacker was no longer just a single fixed probe. It had become a multi-timescale sensing mechanism:

- short windows captured early or burst-local behavior,
- medium windows captured phase-level structure,
- long windows captured whole-workload intensity.

This also helped clarify where the attack was strongest in practical terms. The medium-timescale regime emerged as the best balance between signal strength and invasiveness, while the long-timescale regime remained the strongest purely in terms of separation power.

2.13.8 Intra-Family Evaluation on a QAOA Family Under Static Execution

Once the best attack branch had been identified for the heterogeneous victim set, the next question was whether it could distinguish not just *different classes of algorithms*, but also *different members of the same family*. To test this, the attack was applied to a family of QAOA circuits.

Under static execution, the attack showed that the QAOA family was not invisible or homogeneous. The family fingerprints followed a clear structural progression. Lower-index family members appeared lighter, mid-range members formed tighter clusters, and heavier family members produced stronger waits and larger makespans. However, the attack did not achieve perfect one-by-one identification. Instead, it was best at:

- coarse ordering across the family,
- cluster formation,
- and approximate structural similarity detection.

This was an important result because it showed that the attack could perform intra-family fingerprinting at least at the level of relative structural intensity. In other words, even within a single algorithmic family, the attacker could infer where a circuit lay along a light-to-heavy continuum.

2.13.9 Transition from Static to Sequential and Discovery of a Modeling Mismatch

After these static-attack results had matured, the next natural step was to move the best attack into the sequential execution model. The expectation was that sequential victim execution would introduce an additional source of information: not just aggregate communication intensity, but also stage-local and phase-structured timing signatures.

The first attempt to port the attack to sequential execution appeared to fail. The resulting attacker metrics were almost identical to the static results, which was highly suspicious. This led to a diagnostic phase in which the attack results were compared against the actual sequential baseline statistics. That comparison revealed a strong mismatch. The true sequential-v2 baseline had very different characteristics from the static baseline:

- much smaller average hub waiting time,
- much smaller maximum waiting time,
- a very different number of waited requests,
- and rich stage-level timing statistics.

Therefore, if the sequential attack was producing the same metrics as static, the attack was not actually using the true sequential-v2 semantics. This was an important debugging discovery. It showed that simply labeling the victim trace as sequential was not enough. The sequential-v2 model had to be reproduced more faithfully, especially its stage-by-stage release and drain behavior.

2.13.10 Sequential-v2 Attack Correction

To fix this, the sequential attack driver was rewritten to mirror the true sequential-v2 baseline logic. In this corrected version:

- victim stages were explicitly grouped,
- one stage was processed at a time,
- the hub was fully drained after each stage,
- and an inter-stage gap was introduced, matching the baseline sequential-v2 behavior.

Only after this correction did the attack begin to produce genuinely different behavior from static.

The corrected sequential-v2 attack revealed a different kind of fingerprint. Under static execution, the attack's main information channels were average waiting time, maximum waiting time, and waited fraction. Under sequential-v2, those waiting-based metrics remained useful, but the strongest new signal was *attacker job makespan*. The victim workloads now stretched the attacker's timeline in stage-structured ways. The attacker was no longer observing only how strongly its requests were queued; it was also observing how long the victim's stage-by-stage progression kept its own job active.

This made the sequential-v2 fingerprint richer than the static one. It preserved queue-pressure information but added a second axis: *timeline elongation induced by staged victim execution*. For some workloads, such as DNN, this changed the interpretation significantly. DNN had appeared relatively light in waiting-time space under static execution, but under sequential-v2 it became highly distinguishable because it stretched the attacker's total job duration dramatically. Thus, the sequential-v2 attack provided stronger overall victim separation when richer fingerprints were used.

2.13.11 Intra-Family Evaluation on a QAOA Family Under Sequential-v2

Finally, the best sequential-v2 attack was applied to the same QAOA family. This produced the strongest intra-family fingerprints obtained so far. The family still formed clusters rather than perfectly isolated singletons, but the addition of stage-structured timeline information increased separation considerably. Lower-end QAOA instances remained clearly lighter, heavier ones remained clearly stronger, and the makespan dimension spread the family out more than static execution alone had done.

The result was not perfect exact identification of every member. Neighboring family members with very similar cross-module structure still remained hard to distinguish cleanly. But the attack was now clearly better at:

- ordering QAOA family members,
- identifying cluster membership,
- and inferring relative structural heaviness within the family.

This strengthened the earlier static intra-family result and suggested that sequential-v2 is a more informative environment for attacker fingerprinting, provided that the attack uses a sufficiently rich fingerprint rather than relying only on average waiting time.

2.13.12 Overall Lessons from the Post-Rate-Sweep Refinement

Taken together, the work after the initial probe-rate sweep accomplished several things.

First, it transformed Probe 3 from a promising heuristic into a carefully tuned attacker design with identified best practices:

- dense or medium rate,
- uniform spacing,
- relative-duration normalization,
- and temporal-scale interpretation.

Second, it established that attack quality is not determined solely by how much pressure the attacker applies. In fact, the most effective fingerprinting probe was the one that remained structured, lightweight, and appropriately scaled to victim behavior.

Third, it showed that victim differentiation should be understood through multiple metrics. Waiting time alone is not always sufficient. In static execution, it was the dominant signal. In sequential-v2, makespan and stage-structured elongation became equally important.

Fourth, it demonstrated that the attack can work both across heterogeneous victim classes and within a single algorithm family. The strongest claims are currently at the level of:

- coarse workload class identification,
- relative communication-intensity inference,
- cluster membership,
- and intra-family ordering.

What the attack still does *not* provide is perfect exact algorithm identification or circuit reconstruction. But compared with the initial baseline schedules, the current attack is vastly more informative and far more systematically developed.

2.13.13 Current Best Attack Configurations

At the present stage, the study has effectively converged to two best-attack regimes.

For **static distributed execution**, the strongest branch is:

- Probe 3,
- dense rate (R1),
- uniform spacing,
- relative duration,

- especially medium or long relative windows depending on whether the goal is balanced sensing or maximum separation.

For **sequential-v2 execution**, the strongest branch is:

- the same Probe 3 structure,
- but executed against the true stage-aware victim model,
- with the same relative-duration logic,
- and interpreted using a richer fingerprint that includes both waiting-time metrics and attacker job makespan.

2.13.14 Summary

Since the probe-rate sweep, the attack study has undergone a major progression. It began by fixing the strongest probe-rate candidates, then explored spacing, then absolute and relative duration, then interpreted those durations as short, medium, and long temporal scales. After demonstrating strong victim separation under static execution, the study moved to intra-family evaluation on QAOA circuits. It then diagnosed and corrected a sequential-model mismatch, reconstructed the attack under true sequential-v2 stage-aware execution, and finally extended the intra-family analysis to that setting as well. The end result is a much more mature attack methodology: one that no longer merely demonstrates shared-resource contention, but can now extract meaningful workload fingerprints across algorithms, within algorithm families, and across static versus stage-aware execution models.